

General Training for
Programming Competitions
& Coding Interviews

Winter Semester 2024/2025

SAT Algorithms

Markus Fischer, Paul G. Rump, Christoph Staudt

Friedrich-Schiller-Universität Jena



Boolean Formulas and the Satisfiability Problem

Boolean Formulas

Boolean formulas are logical expressions constructed from binary variables, that is each Variable can be **true** or **false**.

By combining variables with logical operations like **and** and **or** or **negation**, we can express a set of hard constraints in a mathematical formula.

Motivation

Assume that given a set of binary (*true/false* or *yes/no*) constraints, we need to find a valid configuration.

For example: **scheduling tasks** or **allocating resources** without overlaps.

Motivation

Assume that given a set of binary (*true/false* or *yes/no*) constraints, we need to find a valid configuration.

For example: **scheduling tasks** or **allocating resources** without overlaps.

Or assume you are looking for an explanation of a specific outcome of a problem defined by logical rules.

For example in **algorithm analysis** or **software testing**.

Boolean Formulas

Boolean Formulas

A Boolean formulas consists of

- Boolean variables, that can only take values from $\{0,1\}$ (or $\{true, false\}$),
- Binary operators $\wedge$ (**conjunction** / and), $\vee$ (**disjunction** / or),
- the unary operator $\neg$ (**negation**).

Boolean Formulas

A Boolean formulas consists of

- Boolean variables, that can only take values from $\{0,1\}$ (or $\{true, false\}$),
- Binary operators $\wedge$ (**conjunction** / and), $\vee$ (**disjunction** / or),
- the unary operator $\neg$ (**negation**).

Note that $\neg$ bounds stronger than $\wedge$, which bounds stronger than $\vee$:

$$x_1 \vee \neg x_2 \wedge x_3 = x_1 \vee ((\neg x_2) \wedge x_3).$$

Boolean Formulas

A Boolean formulas consists of

- Boolean variables, that can only take values from $\{0,1\}$ (or $\{true, false\}$),
- Binary operators $\wedge$ (**conjunction** / and), $\vee$ (**disjunction** / or),
- the unary operator $\neg$ (**negation**).

Note that $\neg$ bounds stronger than $\wedge$, which bounds stronger than $\vee$:

$$x_1 \vee \neg x_2 \wedge x_3 = x_1 \vee ((\neg x_2) \wedge x_3).$$

A variable x_i or its negation $\neg x_i$ is also called a **literal**.

Boolean Formulas

A Boolean formulas consists of

- Boolean variables, that can only take values from $\{0,1\}$ (or $\{true, false\}$),
- Binary operators $\wedge$ (**conjunction** / and), $\vee$ (**disjunction** / or),
- the unary operator $\neg$ (**negation**).

Note that $\neg$ bounds stronger than $\wedge$, which bounds stronger than $\vee$:

$$x_1 \vee \neg x_2 \wedge x_3 = x_1 \vee ((\neg x_2) \wedge x_3).$$

A variable x_i or its negation $\neg x_i$ is also called a **literal**.

Often boolean formulas are given in **Conjunctive Normal Form** (CNF), that is a conjunction of clauses, where each clause is a disjunction of literals.

Boolean Formulas: Examples

$$f_1(x_1, x_2, x_3) = x_1 \wedge x_2 \vee x_3$$

$$f_2(y_1, y_2, y_3) = \neg(y_1 \vee y_2) \wedge \neg y_3$$

$$f_3(z_1, z_2, z_3) = (z_1 \vee z_2 \vee \neg z_3) \wedge (\neg z_1 \vee z_2)$$

$$f_4(a_1, a_2, a_3) = a_1 \wedge (\neg a_1 \vee a_2) \wedge (\neg a_1 \vee a_3) \wedge (\neg a_2 \vee \neg a_3)$$

$$f_5(b) = b \vee \neg b$$

Boolean Formulas: Examples

$$f_1(x_1, x_2, x_3) = x_1 \wedge x_2 \vee x_3$$

$$f_2(y_1, y_2, y_3) = \neg(y_1 \vee y_2) \wedge \neg y_3$$

$$f_3(z_1, z_2, z_3) = (z_1 \vee z_2 \vee \neg z_3) \wedge (\neg z_1 \vee z_2)$$

$$f_4(a_1, a_2, a_3) = a_1 \wedge (\neg a_1 \vee a_2) \wedge (\neg a_1 \vee a_3) \wedge (\neg a_2 \vee \neg a_3)$$

$$f_5(b) = b \vee \neg b$$

- Which of these are in CNF?

Other Operators

There are even more operators:

- the **implication** $\rightarrow$
- the **logical equivalence** $\leftrightarrow$
- the **exclusive-or (XOR)** $\oplus$

But these can also be expressed using only $\wedge$, $\vee$ and $\neg$. They are mostly used as shorthands to express complicated formulas more succinctly.

Subclasses of CNFs

- Horn-formulas: a CNF where every clause has at most one positive literal:

$$(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee \neg x_3 \vee x_4 \vee \neg x_5)$$

Subclasses of CNFs

- Horn-formulas: a CNF where every clause has at most one positive literal:

$$(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee \neg x_3 \vee x_4 \vee \neg x_5)$$

- 2-CNF: a CNF where every clause has at most two literals:

$$(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee \neg x_3) \wedge (x_4 \vee \neg x_5)$$

Subclasses of CNFs

- Horn-formulas: a CNF where every clause has at most one positive literal:

$$(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee \neg x_3 \vee x_4 \vee \neg x_5)$$

- 2-CNF: a CNF where every clause has at most two literals:

$$(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee \neg x_3) \wedge (x_4 \vee \neg x_5)$$

- 3-CNF: a CNF where every clause has at most three literals:

$$(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee \neg x_3 \vee x_4) \wedge \neg x_5$$

Any boolean formula can be transformed into a 3-CNF-formula.

The Satisfiability Problem

The Satisfiability Problem (SAT)

Given a Boolean formula

$$f : \{0,1\}^n \rightarrow \{0,1\} \text{ in CNF,}$$

is there an $\alpha \in \{0,1\}^n$ such that $f(\alpha) = 1$?

The Satisfiability Problem (SAT)

Given a Boolean formula

$$f : \{0,1\}^n \rightarrow \{0,1\} \text{ in CNF,}$$

is there an $\alpha \in \{0,1\}^n$ such that $f(\alpha) = 1$?

- When talking about Boolean formulas, $\alpha \in \{0,1\}^n$ is often called an **assignment** for f .
- And if $f(\alpha) = 1$, then α is also called a **model** of f .

SAT for Subclasses of CNFs

- Horn-formulas: a CNF where every clause has at most one positive literal.
- 2-CNF: a CNF where every clause has at most two literals.
- 3-CNF: a CNF where every clause has at most three literals.

SAT for Subclasses of CNFs

- Horn-formulas: a CNF where every clause has at most one positive literal.
- 2-CNF: a CNF where every clause has at most two literals.
- 3-CNF: a CNF where every clause has at most three literals.

For Horn-formulas and 2-CNF-formulas, the SAT problem can be solved in **polynomial time**.

However, for 3-CNF-formulas, the SAT problem is **NP-complete**.

Algorithms for SAT

Today, we will look at

- efficient algorithms to solve SAT for Horn-formulas and for 2-CNF-formulas,
- look at some preprocessing steps to simplify a CNF-formula before checking satisfiability,
- and discuss the DPLL-algorithm, which was one of the first algorithms for SAT on general CNFs.

SAT for Horn-CNF

SAT for Horn-Formulas

Horn-formulas consists of clauses that have at most one positive literal, for example:

$$(\neg x_1 \vee \neg x_2 \vee \neg x_3 \vee \neg x_4 \vee y).$$

These clauses can also be written as implications:

$$(x_1 \wedge x_2 \wedge x_3 \wedge x_4) \rightarrow y.$$

SAT for Horn-Formulas

Horn-formulas consists of clauses that have at most one positive literal, for example:

$$(\neg x_1 \vee \neg x_2 \vee \neg x_3 \vee \neg x_4 \vee y).$$

These clauses can also be written as implications:

$$(x_1 \wedge x_2 \wedge x_3 \wedge x_4) \rightarrow y.$$

There are two special cases:

- No positive literal: $(\neg x_1 \vee \neg x_2) \equiv (x_1 \wedge x_2) \rightarrow 0$
- No negative literal: $y \equiv 1 \rightarrow y$.

SAT for Horn-Formulas

This observation allows us to build a directed graph $G = (V, E)$ from the formula with

$$V = \{C \mid C \text{ is a clause of the formula}\} \cup \{0, 1\}$$

$$E = \{(C_1, C_2) \mid \text{there is a variable } x \text{ with } x \in C_1 \text{ and } \neg x \in C_2\}$$

$$\cup \{(1, C) \mid C \text{ is a clause with no negative literal}\}$$

$$\cup \{(C, 0) \mid C \text{ is a clause with no positive literal}\}$$

SAT for Horn-Formulas

This observation allows us to build a directed graph $G = (V, E)$ from the formula with

$$V = \{C \mid C \text{ is a clause of the formula}\} \cup \{0, 1\}$$

$$E = \{(C_1, C_2) \mid \text{there is a variable } x \text{ with } x \in C_1 \text{ and } \neg x \in C_2\} \\ \cup \{(1, C) \mid C \text{ is a clause with no negative literal}\} \\ \cup \{(C, 0) \mid C \text{ is a clause with no positive literal}\}$$

For an example, we consider the Horn-formula:

$$(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_4 \vee x_3) \wedge (\neg x_3 \vee \neg x_5 \vee x_6)$$

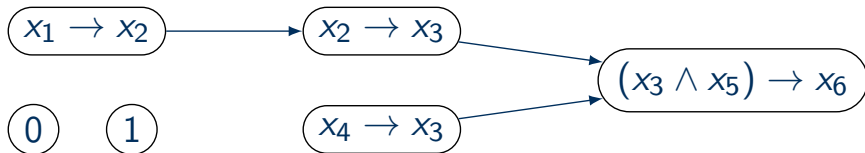
Note that we can also write this as:

$$(x_1 \rightarrow x_2) \wedge (x_2 \rightarrow x_3) \wedge (x_4 \rightarrow x_3) \wedge ((x_3 \wedge x_5) \rightarrow x_6)$$

SAT for Horn-Formulas

$$\begin{aligned} & (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_4 \vee x_3) \wedge (\neg x_3 \vee \neg x_5 \vee x_6) \\ \equiv & (x_1 \rightarrow x_2) \wedge (x_2 \rightarrow x_3) \wedge (x_4 \rightarrow x_3) \wedge ((x_3 \wedge x_5) \rightarrow x_6) \end{aligned}$$

This formula gives the graph:

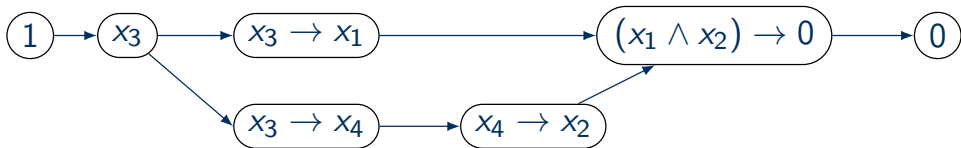


SAT for Horn-Formulas

Another example:

$$\begin{aligned} & (\neg x_1 \vee \neg x_2) \wedge (x_3) \wedge (\neg x_3 \vee x_1) \wedge (\neg x_3 \vee x_4) \wedge (\neg x_4 \vee x_2) \\ \equiv & ((x_1 \wedge x_2) \rightarrow 0) \wedge (x_3) \wedge (x_3 \rightarrow x_1) \wedge (x_3 \rightarrow x_4) \wedge (x_4 \rightarrow x_2) \end{aligned}$$

This formula becomes the following graph:



SAT for Horn-Formulas

Using this graph, we can now check which variables have to be set to 1 to satisfy the formula. Starting with the node 1, we follow the implications and mark all nodes that also have to be 1. If we never come to an implication that leads to the node 0, then the formula is satisfiable.

SAT for Horn-Formulas

Using this graph, we can now check which variables have to be set to 1 to satisfy the formula. Starting with the node 1, we follow the implications and mark all nodes that also have to be 1.

If we never come to an implication that leads to the node 0, then the formula is satisfiable.

In the first example, there is no outgoing edge from node 1. Therefore we can conclude that this formula is satisfiable.

In the second example, when we follow the implications through the graph we learn that zero has to be one, which is a contradiction. This formula is not satisfiable.

SAT for Horn-Formulas: Algorithm

1. Mark the node 1.
2. Initialize a queue that contains the node 1.
3. While there are nodes in the queue:
 - 3.1 get C : the next node from the queue.
 - 3.2 If C is the node 0, return UNSAT
 - 3.3 For all C' that can be reached from C (that is $(C, C') \in E$):
 - 3.4 If every node C'' with $(C'', C') \in E$ is marked,
then mark C' and add C' to the queue.
4. return SAT

A satisfying assignment is constructed by setting all variables that appear as positive literals in a marked clause to 1, and all remaining variables to 0.

SAT for 2-CNF-Formulas

SAT for 2-CNF-Formulas

2-CNFs are different from Horn-formulas, because:

- clauses can only have at most 2 literals, **but**
- any possible combination of positive and negative literals is allowed.

SAT for 2-CNF-Formulas

2-CNFs are different from Horn-formulas, because:

- clauses can only have at most 2 literals, **but**
- any possible combination of positive and negative literals is allowed.

We can check satisfiability for 2-CNF-formulas again with a graph-based approach.

Here we construct the **implication graph** of the formula.

The Implication Graph of a 2-CNF Formula

The implication graph contains a node for every possible literal, that is

$$V = \{x, \neg x \mid x \text{ is a variable in the formula}\}.$$

The Implication Graph of a 2-CNF Formula

The implication graph contains a node for every possible literal, that is

$$V = \{x, \neg x \mid x \text{ is a variable in the formula}\}.$$

And for every clause $(I \vee I')$ (I, I' are literals) we add an edge for each implication that is equal to the clause (note that the clause is equivalent to $(\neg I \rightarrow I')$ and to $(\neg I' \rightarrow I)$).

The Implication Graph of a 2-CNF Formula

The implication graph contains a node for every possible literal, that is

$$V = \{x, \neg x \mid x \text{ is a variable in the formula}\}.$$

And for every clause $(I \vee I')$ (I, I' are literals) we add an edge for each implication that is equal to the clause (note that the clause is equivalent to $(\neg I \rightarrow I')$ and to $(\neg I' \rightarrow I)$).

Therefore, the set of edges for this graph is

$$E = \{(\neg I, I'), (\neg I', I) \mid (I \vee I') \text{ is a clause in the formula}\}.$$

The Implication Graph of a 2-CNF Formula

The implication graph contains a node for every possible literal, that is

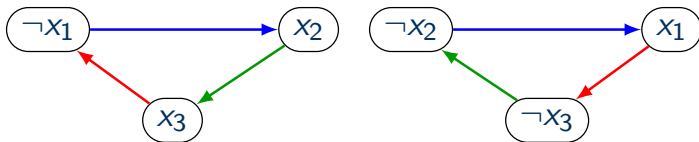
$$V = \{x, \neg x \mid x \text{ is a variable in the formula}\}.$$

And for every clause $(I \vee I')$ (I, I' are literals) we add an edge for each implication that is equal to the clause (note that the clause is equivalent to $(\neg I \rightarrow I')$ and to $(\neg I' \rightarrow I)$).

Therefore, the set of edges for this graph is

$$E = \{(\neg I, I'), (\neg I', I) \mid (I \vee I') \text{ is a clause in the formula}\}.$$

For example, the formula $(x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3 \vee x_1)$ has the implication graph:

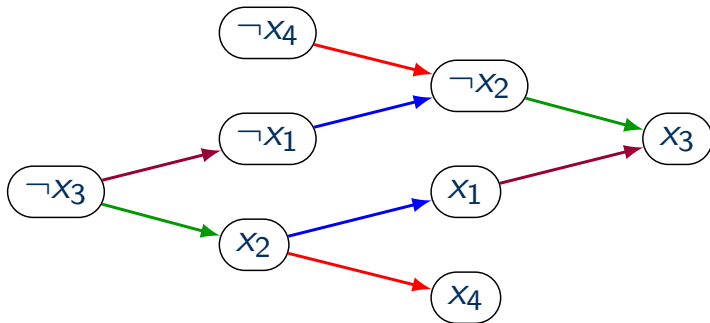


The Implication Graph of a 2-CNF Formula

Another example: we consider the formula

$$(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (\neg x_2 \vee x_4) \wedge (x_3 \vee \neg x_1).$$

The implication graph of this formula is:



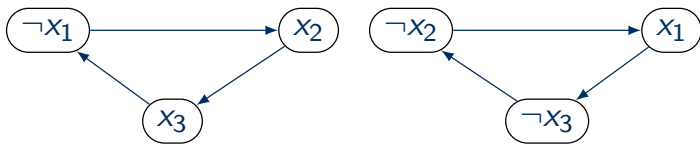
Find Satisfying Assignments with the Implication Graph

The implication graph **encodes all restrictions** on a satisfying assignment for the formula. A directed path from a literal l to a literal l' in the implication graph means that any satisfying assignment that satisfies l also has to satisfy l' .

Find Satisfying Assignments with the Implication Graph

The implication graph **encodes all restrictions** on a satisfying assignment for the formula. A directed path from a literal l to a literal l' in the implication graph means that any satisfying assignment that satisfies l also has to satisfy l' .

Remember the examples from earlier: $(x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3 \vee x_1)$ has the implication graph:

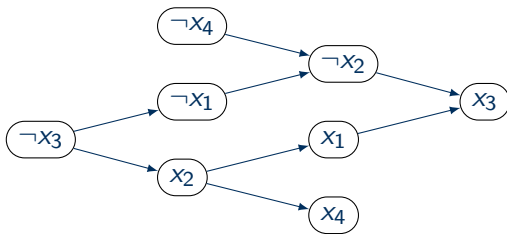


This implication graph is very small, and we can easily see that there are two different assignments that do not lead to a contradiction:

- We can choose $x_1 = 0$, this directly implies $x_2 = 1$ and $x_3 = 1$.
- Or we can choose $x_1 = 1, x_2 = 0, x_3 = 0$.

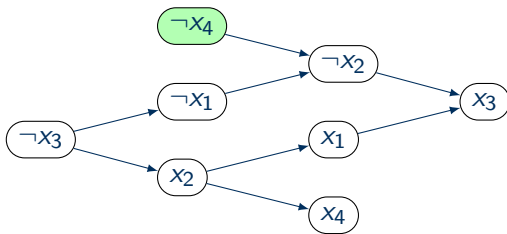
Find Satisfying Assignments with the Implication Graph

The second formula had a more complex implication graph:



Find Satisfying Assignments with the Implication Graph

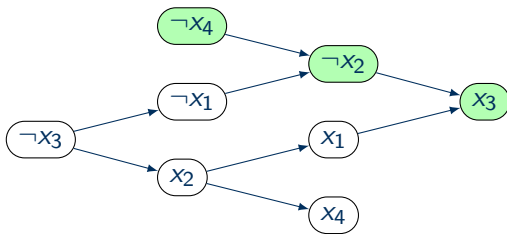
The second formula had a more complex implication graph:



To find a satisfying assignment, we could start with the top node ($\neg x_4$), that is we set $x_4 = 0$.

Find Satisfying Assignments with the Implication Graph

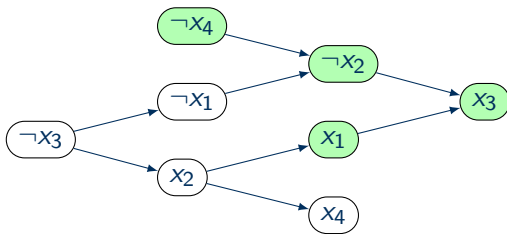
The second formula had a more complex implication graph:



To find a satisfying assignment, we could start with the top node ($\neg x_4$), that is we set $x_4 = 0$. A BFS starting at the node $\neg x_4$ (following only directed edges) tells us that we have to set $x_2 = 0$ and $x_3 = 1$.

Find Satisfying Assignments with the Implication Graph

The second formula had a more complex implication graph:

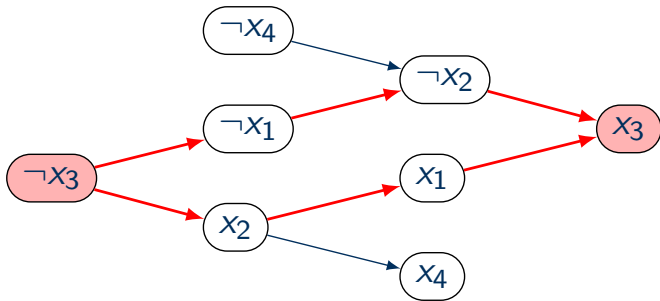


To find a satisfying assignment, we could start with the top node ($\neg x_4$), that is we set $x_4 = 0$. A BFS starting at the node $\neg x_4$ (following only directed edges) tells us that we have to set $x_2 = 0$ and $x_3 = 1$.

It only remains to choose a value for x_1 , for example set $x_1 = 1$. Another BFS starting at x_1 does not lead to a contradiction with any of the already set variables, so we have found a satisfying assignment.

Implication Graphs of Unsatisfiable Formulas

Note that we cannot set $x_3 = 0$. A BFS starting at node $\neg x_3$ reaches the node x_3 , which means that no satisfying assignment can have $x_3 = 0$:

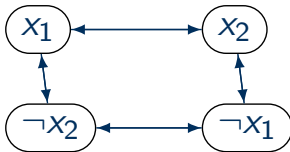


However, we can choose $x_3 = 1$, because there is no directed path from x_3 to $\neg x_3$.

Not Satisfiable Formulas

If the implication graph contains a directed path from a literal l to its negation $\neg l$ and also from $\neg l$ to l , then the formula has no satisfying assignment.

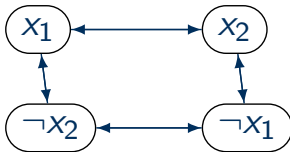
For example, the formula $(\neg x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_2) \wedge (x_1 \vee x_2)$ has the implication graph:



Not Satisfiable Formulas

If the implication graph contains a directed path from a literal l to its negation $\neg l$ and also from $\neg l$ to l , then the formula has no satisfying assignment.

For example, the formula $(\neg x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_2) \wedge (x_1 \vee x_2)$ has the implication graph:



A set of nodes where each node can reach any other node via a directed path is called a **strongly connected component**.

SAT for 2-CNF-Formulas

Therefore, the following algorithm can decide if a 2-CNF-formula is satisfiable:

1. Construct the implication graph of the formula.
2. For each strongly connected component in the implication graph:
 - 2.1 check that at most one literal from each variable appears in the component.

SAT for 2-CNF-Formulas

Therefore, the following algorithm can decide if a 2-CNF-formula is satisfiable:

1. Construct the implication graph of the formula.
2. For each strongly connected component in the implication graph:
 - 2.1 check that at most one literal from each variable appears in the component.

To find the strongly connected components of the graph, you can use Tarjan's algorithm¹.

Or check the reachability directly using BFS for each literal.

¹for example

https://en.wikipedia.org/wiki/Tarjan's_strongly_connected_components_algorithm

The DPLL-Algorithm

SAT Preprocessing: Unit Clause Propagation

Consider the following CNF formula:

$$(x_1 \vee x_2) \wedge (x_3) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge (x_1 \vee \neg x_4 \vee \neg x_3) \wedge (x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_4).$$

This seems like a complex CNF, and we cannot decide at first glance if the formula is satisfiable or not.

SAT Preprocessing: Unit Clause Propagation

Consider the following CNF formula:

$$(x_1 \vee x_2) \wedge (x_3) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge (x_1 \vee \neg x_4 \vee \neg x_3) \wedge (x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_4).$$

This seems like a complex CNF, and we cannot decide at first glance if the formula is satisfiable or not.

However, note that there is a clause that contains only one literal: (x_3) . Therefore any satisfying assignment has to set $x_3 = 1$.

SAT Preprocessing: Unit Clause Propagation

Consider the following CNF formula:

$$(x_1 \vee x_2) \wedge (x_3) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge (x_1 \vee \neg x_4 \vee \neg x_3) \wedge (x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_4).$$

This seems like a complex CNF, and we cannot decide at first glance if the formula is satisfiable or not.

However, note that there is a clause that contains only one literal: (x_3) . Therefore any satisfying assignment has to set $x_3 = 1$.

This observation allows us to simplify the CNF as follows:

- Any clause that contains the literal x_3 can be removed, because x_3 has to be 1 and therefore the **clause is satisfied**.
- From any clause that contains the literal $\neg x_3$, the literal can be removed, because this literal will **never help to satisfy** the clause.

SAT Preprocessing: Unit Clause Propagation

Consider the following CNF formula:

$$(x_1 \vee x_2) \wedge (x_3) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge (x_1 \vee \neg x_4 \vee \neg x_3) \wedge (x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_4).$$

This seems like a complex CNF, and we cannot decide at first glance if the formula is satisfiable or not.

However, note that there is a clause that contains only one literal: (x_3) . Therefore any satisfying assignment has to set $x_3 = 1$.

This observation allows us to simplify the CNF as follows:

- Any clause that contains the literal x_3 can be removed, because x_3 has to be 1 and therefore the **clause is satisfied**.
- From any clause that contains the literal $\neg x_3$, the literal can be removed, because this literal will **never help to satisfy** the clause.

Therefore the simplified CNF looks like this:

$$(x_1 \vee x_2) \wedge (x_1 \vee \neg x_4) \wedge (x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_4).$$

This procedure is called *unit clause propagation* (UCP).

SAT Preprocessing: Unit Clause Propagation

Unit Clause Propagation for a CNF formula φ :

1. While there is a clause C in φ that contains only one literal l :
 - 1.1 Remove every clause C' from φ that contains the literal l .
 - 1.2 For every clause C' that contains the literal $\neg l$: remove $\neg l$ from C' .
 - 1.3 If φ now contains an empty clause, return **not satisfiable**.
2. If φ contains no clauses, return **satisfiable**.

SAT Preprocessing: Pure Literal Elimination

Continuing with the example, there is even more that we can do to simplify the formula:

$$(x_1 \vee x_2) \wedge (x_1 \vee \neg x_4) \wedge (x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_4).$$

SAT Preprocessing: Pure Literal Elimination

Continuing with the example, there is even more that we can do to simplify the formula:

$$(x_1 \vee x_2) \wedge (x_1 \vee \neg x_4) \wedge (x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_4).$$

Note that x_1 only appears as positive literal. Therefore we can set $x_1 = 1$, no clause will become unsatisfiable by this step.

SAT Preprocessing: Pure Literal Elimination

Continuing with the example, there is even more that we can do to simplify the formula:

$$(x_1 \vee x_2) \wedge (x_1 \vee \neg x_4) \wedge (x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_4).$$

Note that x_1 only appears as positive literal. Therefore we can set $x_1 = 1$, no clause will become unsatisfiable by this step.

All clauses that contain x_1 can be removed from the formula, and it remains

$$(x_2 \vee x_4).$$

This procedure is called *pure literal elimination* (PLE).

SAT Preprocessing: Pure Literal Elimination

Pure Literal Elimination for a CNF formula φ :

1. For every variable x in φ :
 - 1.1 if x appears only in positive literals or only in negative literals, then remove all clauses that contain x from φ .
 - 1.2 If φ contains no clauses, return **satisfiable**.

SAT Preprocessing: Pure Literal Elimination

Pure Literal Elimination for a CNF formula φ :

1. For every variable x in φ :
 - 1.1 if x appears only in positive literals or only in negative literals, then remove all clauses that contain x from φ .
 - 1.2 If φ contains no clauses, return **satisfiable**.

Note that after applying UCP or PLE, new opportunities for simplification can arise. We have seen that in the example: the pure literal elimination of the literal x_1 became only possible after the unit clause propagation of x_3 .

SAT for Generic CNF-Formulas

Deciding satisfiability for 3-CNF formulas (or CNF formulas in general) is still an active field of research. We will briefly discuss one of the very first SAT-solvers: the Davis–Putnam–Logemann–Loveland algorithm (DPLL) from 1961.

SAT for Generic CNF-Formulas

Deciding satisfiability for 3-CNF formulas (or CNF formulas in general) is still an active field of research. We will briefly discuss one of the very first SAT-solvers: the Davis–Putnam–Logemann–Loveland algorithm (DPLL) from 1961.

The basic idea is very simple: divide-and-conquer, combined with preprocessing.

1. Simplify the CNF using with UCP and PLE.
2. If the simplified CNF is trivially satisfiable, return `true`.
3. If the simplified CNF is trivially not satisfiable, return `false`.
4. Choose a variable from the CNF, and construct the two CNFs where the variable is once set to 1, and once set to 0.
5. Run DPLL recursively on both of the constructed CNFs, return `true` if one of them is satisfiable, otherwise return `false`.

The DPLL Algorithm

In each recursive call, the algorithm only removes one variable from the formula. Therefore in the worst case, the algorithm still has to **check exponentially many assignments**.

The DPLL Algorithm

In each recursive call, the algorithm only removes one variable from the formula. Therefore in the worst case, the algorithm still has to **check exponentially many assignments**.

However, the power of the algorithm lies in the **preprocessing**. Ideally, additional variables and a large set of clauses can be removed by UCP and PLE. In practice, the CNF becomes significantly smaller with each recursive call.

The DPLL Algorithm: Example

Let us consider the following 3-CNF-formula:

$$\begin{aligned} & (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_4 \vee x_5) \wedge (\neg x_1 \vee x_4 \vee \neg x_5) \wedge (\neg x_1 \vee \neg x_2 \vee x_4) \\ & \wedge (\neg x_1 \vee x_3 \vee x_5) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_2 \vee x_4 \vee \neg x_5) \\ & \wedge (x_3 \vee \neg x_4 \vee x_5) \wedge (\neg x_4 \vee x_5) \end{aligned}$$

The DPLL Algorithm: Example

Let us consider the following 3-CNF-formula:

$$\begin{aligned} & (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_4 \vee x_5) \wedge (\neg x_1 \vee x_4 \vee \neg x_5) \wedge (\neg x_1 \vee \neg x_2 \vee x_4) \\ & \wedge (\neg x_1 \vee x_3 \vee x_5) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_2 \vee x_4 \vee \neg x_5) \\ & \wedge (x_3 \vee \neg x_4 \vee x_5) \wedge (\neg x_4 \vee x_5) \end{aligned}$$

There is no possibility for simplification with UCP or PLE here. Therefore we have to choose a variable and start the recursion, we start with x_1 and set $x_1 = 1$.

The DPLL Algorithm: Example

Let us consider the following 3-CNF-formula:

$$\begin{aligned} & (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_4 \vee x_5) \wedge (\neg x_1 \vee x_4 \vee \neg x_5) \wedge (\neg x_1 \vee \neg x_2 \vee x_4) \\ & \wedge (\neg x_1 \vee x_3 \vee x_5) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_2 \vee x_4 \vee \neg x_5) \\ & \wedge (x_3 \vee \neg x_4 \vee x_5) \wedge (\neg x_4 \vee x_5) \end{aligned}$$

There is no possibility for simplification with UCP or PLE here. Therefore we have to choose a variable and start the recursion, we start with x_1 and set $x_1 = 1$. This satisfies the first two clauses and simplifies three more:

$$\begin{aligned} & (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_4 \vee x_5) \wedge (\neg x_1 \vee x_4 \vee \neg x_5) \wedge (\neg x_1 \vee \neg x_2 \vee x_4) \\ & \wedge (\neg x_1 \vee x_3 \vee x_5) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_2 \vee x_4 \vee \neg x_5) \\ & \wedge (x_3 \vee \neg x_4 \vee x_5) \wedge (\neg x_4 \vee x_5) \end{aligned}$$

The DPLL Algorithm: Example

Now we can simplify the CNF further using UCP and PLE. There are no unit clauses, but there are pure literals: x_2 only appears negated in the remaining formula.

The DPLL Algorithm: Example

Now we can simplify the CNF further using UCP and PLE. There are no unit clauses, but there are pure literals: x_2 only appears negated in the remaining formula.

Therefore we can set $x_2 = 0$ and remove all clauses that contain $\neg x_2$:

$$\begin{aligned} & (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_4 \vee x_5) \wedge (\neg x_1 \vee x_4 \vee \neg x_5) \wedge (\neg x_1 \vee \neg x_2 \vee x_4) \\ & \wedge (\neg x_1 \vee x_3 \vee x_5) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_2 \vee x_4 \vee \neg x_5) \\ & \wedge (x_3 \vee \neg x_4 \vee x_5) \wedge (\neg x_4 \vee x_5) \end{aligned}$$

The DPLL Algorithm: Example

Now we can simplify the CNF further using UCP and PLE. There are no unit clauses, but there are pure literals: x_2 only appears negated in the remaining formula.

Therefore we can set $x_2 = 0$ and remove all clauses that contain $\neg x_2$:

$$\begin{aligned} & (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_4 \vee x_5) \wedge (\neg x_1 \vee x_4 \vee \neg x_5) \wedge (\neg x_1 \vee \neg x_2 \vee x_4) \\ & \wedge (\neg x_1 \vee x_3 \vee x_5) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_2 \vee x_4 \vee \neg x_5) \\ & \wedge (x_3 \vee \neg x_4 \vee x_5) \wedge (\neg x_4 \vee x_5) \end{aligned}$$

Now there is a new pure literal: x_3 only appears as positive literal.

The DPLL Algorithm: Example

Now there is a new pure literal: x_3 only appears as positive literal.

Therefore we can set $x_3 = 1$ and remove all clauses that contain x_3 :

$$\begin{aligned} & (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_4 \vee x_5) \wedge (\neg x_1 \vee x_4 \vee \neg x_5) \wedge (\neg x_1 \vee \neg x_2 \vee x_4) \\ & \wedge (\neg x_1 \vee x_3 \vee x_5) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_2 \vee x_4 \vee \neg x_5) \\ & \wedge (x_3 \vee \neg x_4 \vee x_5) \wedge (\neg x_4 \vee x_5) \end{aligned}$$

The DPLL Algorithm: Example

Now there is a new pure literal: x_3 only appears as positive literal.

Therefore we can set $x_3 = 1$ and remove all clauses that contain x_3 :

$$\begin{aligned} & (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_4 \vee x_5) \wedge (\neg x_1 \vee x_4 \vee \neg x_5) \wedge (\neg x_1 \vee \neg x_2 \vee x_4) \\ & \wedge (\neg x_1 \vee x_3 \vee x_5) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_2 \vee x_4 \vee \neg x_5) \\ & \wedge (x_3 \vee \neg x_4 \vee x_5) \wedge (\neg x_4 \vee x_5) \end{aligned}$$

Now we could recurse on this formula, or in this case solve it using one of the algorithms for 2-CNFs, Horn-formulas, or using brute force since only two variables remain.

Exercise

Exercise

- Solve the SAT-solver puzzle at lightoj:
<https://lightoj.com/problem/forming-the-council>

Exam Questions

Exam Questions

1. None!